《记得喊我录播》

代理模式（结构型）

动态代理：

jdk: 被代理的类，不能再被代理，接口可以被代理

jdk生成的代理类会去实现被代理类实现的接口

jdk通过反射去调用目标方法，调用时慢（相对而言），生成动态代理快（因为只有一个类），

cglib:

生成的代理类会去继承被代理类

fastClass机制，给每一个方法生成一个index索引，通过索引直接定位到方法

调用时快（根据index找到方法），生成时慢（生成了很多类）

什么样的类不能被代理

final修饰的类（cglib）

被代理的类

方法是非public

装饰器模式（结构型）

一定有一个构造函数的参数为当前类实现的接口（继承）类参数

```
BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(new
InputStream()));
bufferedReader.readLine();
```

委派模式（行为）

策略模式（行为）

观察者模式（行为）

1.给obs的list中放观察者对象

2.去发布我们问题

3.发布问题的时候是去obs中查找观察者，然后调用观察者的update()

一种是jdk提供的

被观察者必须继承Observable类

观察者必须实现Observer接口

被观察者要去setChange()改变状态，然后调用notifyObservers方法

观察者要实现update

一种是guava提供的

只需要定义消息总线EventBus

然后注册事件到EventBus上

事件中的方法添加@Subscribe注解就可以（温馨提示：放入入参不要用基本类型，要用封装类型）

建造者模式（创建）

```
        List<String> builder1 = new ValidateService().validate1()
                .validate2()
                .validate3()
                .validate4()
                .builder();
```

```
public class ValidateService {

    private List<String> list = new ArrayList<String>();

    public ValidateService validate1(){
//        if("".equals(person.getName())){
//            list.add("名字不能为空");
//        }
        return this;
    }

    public ValidateService validate2(){
        list.add("error");
        return this;
    }

    public ValidateService validate3(){
        list.add("error");
        return this;
    }
```

```
    public ValidateService validate4(){
        return this;
    }

    public ValidateService validate5(){
        return this;
    }

    public List<String> builder(){
        return list;
    }

}
```

三大设计模式类型

设计模式的的3大类型其实是一个递进的过程，主要是基于它这个设计模式的目的

首先，你要实现功能，肯定要创建对象是不是，所以像单例、工厂、建造者、原型都属于怎么去很好的创建对象

创建完对象后是不是就应该考虑对象之间怎么合理的存在，如何更好的继承、依赖、组合，那么就衍生了很多结构型的模式、比如门面、适配器、代理、装饰、组合、享元

前面2步做完，就是到了具体的实现了。怎么更好的达到目的，那么为了行为更清晰，效率更高就是行为型模式，比如我们的委派，后面的策略，责任链、迭代器等等

为什么要有设计模式?

1.为了减少代码量（错）

2.可以提高代码的可扩展性

3.设计模式便于理解代码（错）

单例

工厂

原型

深克隆

递归，json ,序列化

浅克隆

clonable

深克隆和浅克隆的区别呢?

浅克隆是引用

深克隆是一个新的对象